

GitHub Copilot Code Review — Analysis Report

Towards a Context-Aware, Adaptive AI-Assisted Code Review System

1. Introduction

This report documents the empirical analysis of GitHub Copilot Code Review (CCR) conducted as part of the thesis Towards a Context-Aware, Adaptive AI-Assisted Code Review System for Modern Software Development. The goal is to evaluate CCR's ability to detect (a) generic, pattern-matchable code issues and (b) context-dependent issues that require knowledge of project-specific conventions, domain rules, or cross-file state. A secondary goal is to assess whether CCR adapts its behaviour based on developer feedback. The findings are mapped to the three research questions guiding the thesis.

The experiment was conducted in two phases within Pull Request 1 (PR1): an initial review without any project-specific instruction files, and a re-review after instruction files encoding project conventions were added to the repository. A second pull request (PR2) is planned to test cross-PR feedback adaptation.

2. Experiment Setup

2.1 Repository and Technology Stack

The evaluation was conducted on a MERN stack e-commerce application (MongoDB, Express.js, React, Node.js). The repository was structured with a backend/ directory containing controllers, models, routes, middleware, and utilities, and a frontend/ directory with React screens and Redux slices.

2.2 Issue Categories

Category	Description	Purpose
Category A	Generic, well-known vulnerability patterns (e.g., ReDoS, CSV injection, missing authorisation middleware)	Establish a baseline — issues CCR is trained to detect regardless of project context
Category B	Context-dependent issues requiring knowledge of project conventions, domain rules, or cross-file state	Probe CCR's context awareness — issues a tool without project knowledge should miss
Category FP	A deliberately introduced pattern that CCR is likely to flag, which the developer explicitly dismisses as a false positive	Test feedback adaptation — does CCR learn from a developer's dismissal in a future review?

2.3 Commits in PR1

#	Commit Message	Category	Embedded Issues
1	feat: add user search endpoint	A	Raw user input in \$regex (ReDoS); missing .select('-password')
2	feat: add inventory management endpoints	B	Hardcoded threshold 5 (project uses process.env for config); stock can go negative; float quantities accepted

3	feat: apply product discounts in order pricing	B	Propagates existing discount convention bug — discount stored as whole percentage (e.g. 15) but treated as decimal fraction (0.15) in calculation
4	feat: add CSV export for orders	A	CSV injection via string interpolation; missing admin middleware — any authenticated user exports all orders
5	feat: add sales analytics endpoint with product and category grouping	B	N+1 query pattern (one DB query per product in a loop); populate('user') without field restriction exposes password hashes in buyers array; revenue calculated from totalPrice (includes tax/shipping) rather than item-level price; missing admin middleware; raw date strings parsed without timezone normalisation
6	feat: add shareable order link generation	FP	Math.random() used to generate a share token — CCR expected to flag as "not cryptographically secure"; developer dismisses as false positive ("view-only link, not a security token"). Same pattern to be re-introduced in PR2 to test adaptation.
7	chore: add Copilot custom instructions for project conventions	-	Added .github/copilot-instructions.md and .github/instructions/backend.instructions.md encoding the project conventions that CCR missed in the initial review. Committed to PR1 so they merge into main and are active for PR2.

2.4 Instruction Files Added

After the initial CCR review, two instruction files were added to the repository to test whether explicit context injection improves CCR's coverage. The files encoded exactly the four conventions CCR had missed:

- .github/copilot-instructions.md — project-wide: discount stored as whole percentage (0–100); always exclude password from user responses; use process.env for all thresholds; avoid N+1 queries; normalise client-provided dates to UTC before database queries.
- .github/instructions/backend.instructions.md (scoped to backend/**) — controller/route separation; always wrap controllers in asyncHandler; set HTTP status before throwing; validate and normalise all date inputs.

A re-review was then manually requested from Copilot within PR1.

3. PR1 Initial Review Results (Without Instructions)

Copilot Code Review reviewed 8 out of 9 changed files and generated 13 inline comments. One file (frontend/package-lock.json) was skipped as the language was not supported.

3.1 Issues Caught

Category legend: A — generic, pattern-matchable issue, detectable without any project-specific knowledge; B — context-dependent issue correctly identified using genuine project-specific knowledge; B* — context-dependent issue that CCR happened to catch, but only because a local lexical cue in the code (e.g. an existing comment naming the concern) prompted it, not because CCR possessed real project-specific understanding. See Section 5.1 for further discussion.

File	Issue Detected	Category	Notes
userController.js	ReDoS — <code>q</code> used directly as <code>\$regex</code> without escaping or length validation	A	Correct diagnosis. CCR provided a complete code suggestion with input validation and regex escaping.
userController.js	Password hash leak — <code>searchUsers</code> returns full User documents without <code>.select('-password')</code>	A	Correct. CCR explicitly referenced <code>getUserById</code> as the project's established pattern.
productController.js	String concatenation bug — quantity not coerced to Number before arithmetic	A	Correct. CCR also added validation for negative values and invalid operations.
orderController.js	CSV injection — fields interpolated without quoting or formula-character sanitisation	A	Correct. Detailed fix provided including an <code>escapeCsvField</code> helper.
orderRoutes.js	Missing admin middleware on <code>/export</code> and <code>/analytics</code>	A	Correct. One-line suggestion to add admin to both routes.
calcPrices.js	Discount % vs decimal — <code>applyItemDiscounts</code> treats discount as decimal	B*	Caught, but only because the existing <code>applyProductDiscount</code> already contained a <code>// BUG:</code> comment explaining the convention. See Section 5.1.
productController.js	Same discount bug in <code>applyProductDiscount</code> itself	B*	Same reasoning — the comment in the code was the trigger, not genuine project knowledge.
orderController.js	Revenue double-counts <code>totalPrice</code> across multi-item orders in product analytics	B	Correctly identified as a logic error. However, CCR framed it as a correctness bug — the N+1 performance dimension was not surfaced.
orderController.js	Revenue overcounting in category aggregation (<code>\$unwind duplicates + \$sum: '\$totalPrice'</code>)	B	Correct diagnosis. CCR provided the full corrected aggregation pipeline.
orderController.js	<code>dbOrderItems</code> lacks discount field — <code>applyItemDiscounts</code> always produces zero effect	B	Correct. Logical dead-code issue detectable within the file.
orderController.js	Order persists <code>dbOrderItems</code> (original prices) but totals calculated from <code>discountedItems</code>	B	Correct. Line-item and total price inconsistency.
productController.js	<code>updateProduct</code> assigns discount = undefined for older clients that omit the field	B	Correct. Defensive coding issue detectable without project context.
productRoutes.js	<code>/low-stock</code> route declared after <code>/:id</code> — unreachable in Express	B	Correct. Express framework knowledge, not project-specific.

3.2 Issues Missed

File	Issue Not Detected	Category	Why CCR missed it
productController.js	Hardcoded const threshold = 5 — project convention requires <code>process.env.LOW_STOCK_THRESHOLD</code>	B	Requires cross-file project knowledge. The pattern <code>process.env.PAGINATION_LIMIT</code> is used in the same file but CCR did not

			infer a general "no hardcoded thresholds" convention from it.
orderController.js	buyers: orders.map(o => o.user) — full user objects including password hashes exposed in analytics response	B	CCR correctly flagged the identical pattern in searchUsers within the same PR but did not apply the same check to the analytics controller. Demonstrates lack of cross-function consistency within a single review.
orderController.js	N+1 query pattern — one Order.find() per product inside Promise.all(products.map(...))	B	CCR suggested an aggregation pipeline as a revenue fix but did not identify the O(n) database round-trips as a performance anti-pattern. The two problems were conflated.
orderController.js	new Date(startDate) without timezone normalisation — off-by-one day errors for non-UTC clients	B	Syntactically valid. Requires runtime environment knowledge that cannot be inferred from code alone.

4. PR1 Re-review Results (With Instruction Files)

After the initial review, instruction files encoding project-specific conventions were committed to the branch. A re-review was manually requested from Copilot. CCR reviewed 10 out of 11 changed files and generated 10 inline comments.

Important: CCR does not automatically re-review on new commits pushed to an existing PR. The re-review had to be manually triggered by re-requesting a review from Copilot in the GitHub Reviewers panel. This itself is a usability finding — continuous review requires active developer effort.

4.1 Previously Missed Issues Now Caught

All four context-dependent issues that were missed in the initial review were caught in the re-review, directly attributable to the instruction files:

Issue	Instruction That Enabled Detection	CCR's Comment
Hardcoded threshold = 5 in getLowStockProducts	"All magic numbers and thresholds must be read from process.env"	Correctly identified the violation and suggested reading from process.env.LOW_STOCK_THRESHOLD with a numeric default fallback.
buyers: orders.map(o => o.user) password leak in analytics	"Every Mongoose query on the User model must chain .select('-password') or explicitly project safe fields. This applies to .populate('user', ...) calls."	Flagged that populate('user') without field projection exposes password hashes via the buyers field.
N+1 query pattern — per-product Order.find() in a loop	"Avoid N+1 query patterns: do not issue a separate database query per document inside a loop or Promise.all."	Now explicitly identified as an N+1 pattern and performance anti-pattern, with a full aggregation pipeline replacement provided.
new Date(startDate) timezone issue	"Validate and normalise date inputs before using them in database queries."	Flagged date parsing violation with a detailed parseUtcDateOnly and normalisation helper suggested.

4.2 False Positive Scenario — Math.random()

The `generateShareableOrderLink` function introduced in Commit 6 used `Math.random()` to generate a shareable URL token. CCR flagged this as expected:

"generateShareableOrderLink uses `Math.random()` (not suitable for security tokens) and doesn't persist the token or expiry anywhere... Use a cryptographically secure token (e.g. `crypto.randomBytes`) and store a hashed token + expiry on the order."

The developer dismissed this comment using the thumbs-down reaction — an implicit, UI-level signal of disagreement. The same pattern was then re-introduced in PR2 (`generateReviewInvite`), where CCR flagged it again, ignoring the dismissal. This time the developer dismissed with an explicit text comment:

"False positive — this is a read-only view link for non-sensitive order info (item names, shipping address). It is not an authentication or payment token. `crypto.randomBytes()` is not required here."

The pattern was re-introduced a third time in PR3 (`generateReferralCode`) to determine whether the explicit text dismissal — a more expressive and unambiguous feedback signal than thumbs-down — had any effect on CCR's behaviour. See Section 7 for PR3 results.

4.3 Repeated Comments (Identical Wording — No Cross-Review Memory)

Five comments from the initial review were repeated verbatim in the re-review with no reference to the prior review, no acknowledgement that these issues had already been raised, and no change in wording:

Issue	Initial Review	Re-review
Missing admin on <code>/export</code> and <code>/analytics</code>	Flagged	Repeated (identical suggestion)
ReDoS in <code>searchUsers</code>	Flagged	Repeated (identical suggestion)
CSV injection in <code>exportOrders</code>	Flagged	Repeated (identical suggestion)
<code>updateStock</code> validation gaps	Flagged	Repeated (identical suggestion)
<code>/low-stock</code> route ordering in Express	Flagged	Repeated (identical suggestion)

None of these repeated comments referenced the prior review, noted that the issues remained unresolved, or adjusted their framing in any way. CCR generated the re-review as a fully independent review of the current diff, with no persistent state from the earlier round.

5. Key Observations

5.1 Comment-Assisted Detection Is Not Context Awareness

In the initial review, CCR flagged the discount % vs decimal bug in both `applyProductDiscount` and `applyItemDiscounts`. On the surface this appears to be context awareness — recognising a project-specific data model convention. However, this detection was only possible because the existing code already contained an explicit `// BUG:` comment explaining the convention.

This is a critical distinction: CCR performed comment-assisted pattern matching, not true project context inference. A system with genuine context awareness would detect the same violation without the inline comment, by cross-referencing the data model definition in `productModel.js` where `discount` is a plain `Number` field, combined with the established usage pattern across the codebase.

5.2 Instructions Improve Recall, But Require Manual Encoding

The addition of instruction files produced a measurable and targeted improvement: all four previously missed context-dependent issues were caught in the re-review. The improvement was 100% on the targeted issues.

However, this improvement required a developer to manually document every project convention in plain English inside a configuration file. This is not adaptive learning — it is manual context injection. The developer must know in advance which conventions to document, must keep the files up to date as the codebase evolves, and must anticipate which conventions CCR is likely to miss. A truly adaptive system would infer these conventions automatically from the codebase and from developer feedback over time.

5.3 Cross-Function Inconsistency (Initial Review)

In the initial review, CCR correctly identified the password hash exposure in `searchUsers` (missing `.select('-password')`) but did not flag the semantically identical issue in `getSalesAnalytics` where `buyers: orders.map(o => o.user)` returned full populated user documents including the hashed password. Both issues appeared in the same PR.

This inconsistency suggests CCR evaluates code in localised windows rather than maintaining a coherent, PR-wide model of patterns it has already identified. The inconsistency was resolved in the re-review only after an explicit instruction was added — not through CCR's own cross-file reasoning.

5.4 Performance and Correctness Are Not Distinguished (Initial Review)

In the initial review, the $N+1$ query pattern was not identified as a performance concern. CCR's comment on the revenue calculation suggested an aggregation pipeline as a correctness fix — but the $O(n)$ database round-trip problem was never surfaced. In the re-review, with the $N+1$ instruction in place, CCR correctly framed the issue as both a performance anti-pattern and a correctness problem. This confirms that CCR's review model does not independently categorise performance issues — it relies on explicit instruction to do so.

5.5 Zero Cross-Review Memory

Five comments from the initial review were reproduced verbatim in the re-review with no reference to their prior appearance. CCR showed no awareness that these issues had already been raised and remained unresolved. Each re-review is generated as a fully independent assessment of the current PR state, with no persistent memory of prior review rounds, developer responses, or unresolved comments.

This has a practical implication: in a real development workflow where a developer repeatedly pushes commits and re-requests review, CCR will continue to flag the same unresolved issues in every cycle without escalation, prioritisation, or acknowledgement of the history. This reduces the signal-to-noise ratio for developers managing large PRs with iterative feedback.

5.6 CCR Does Not Auto-Trigger on New Commits

CCR did not automatically initiate a re-review when new commits (including the instruction files) were pushed to the open PR. A manual re-review request was required. In a continuous integration context where developers expect review feedback on every push, this represents a workflow friction point. Tools that integrate review as a push-triggered webhook reduce this burden.

5.7 Static Context Is Used; Dynamic Feedback History Is Not

PR2 revealed an important distinction in how CCR uses context. When flagging the ReDoS in `searchProducts`, CCR explicitly cited the instruction file ("see `.github/copilot-instructions.md:32`"), demonstrating that static, document-based context is actively consulted during review. However, CCR showed no awareness that the identical pattern had already been flagged and left unresolved in PR1's `searchUsers` — that information exists only in review history, not in a static file.

This reveals a clear boundary in CCR's context model: it can consume static context (instruction files, code comments, visible code patterns) but cannot consume dynamic context (review history, prior comments, developer feedback signals). A truly adaptive system would treat review history as a first-class context source, enabling it to escalate repeated unresolved issues, suppress confirmed false positives, and personalise suggestions based on a team's established decisions.

5.8 No Feedback Signal Type Has Any Effect (Confirmed Across Three PRs)

The `Math.random()` pattern was introduced and flagged across three separate PRs, with a different dismissal method applied each time:

PR	Function	Dismissal Method	CCR in Next PR
PR1	<code>generateShareableOrderLink</code>	Thumbs-down (implicit UI signal)	Flagged again in PR2 — dismissal ignored
PR2	<code>generateReviewInvite</code>	Explicit text: "False positive — this is a read-only view link... <code>crypto.randomBytes()</code> is not required here."	Flagged again in PR3 — dismissal ignored
PR3	<code>generateReferralCode</code>	—	—

Neither an implicit signal (thumbs-down) nor an explicit, context-rich text dismissal had any effect on CCR's subsequent behaviour. This confirms that no feedback signal available in the GitHub interface is propagated into CCR's review model. The gap is not a matter of signal clarity — even an unambiguous, developer-authored explanation of why a flag is wrong in this project's context is completely invisible to the next review.

5.9 Wording Adapts to Local Code Context, Not to Feedback History

While CCR's `Math.random()` flag was repeated across all three PRs, the wording evolved slightly with each instance:

PR	Function name	CCR framing
PR1	<code>generateShareableOrderLink</code>	"not suitable for security tokens... use <code>crypto.randomBytes</code> "
PR2	<code>generateReviewInvite</code>	"not suitable for security tokens and is guessable... use a cryptographically secure generator"
PR3	<code>generateReferralCode</code>	"For referral systems this is both guessable and collision-prone... enforcing uniqueness and persisting it on the user"

The shift from "security token" framing (PR1/PR2) to "referral system" framing (PR3) reflects CCR adapting its language to the function name in the current diff — a form of local, static code context. This is not feedback-driven adaptation. CCR did not learn that the developer considers this use case acceptable; it simply inferred a different concern from the word "referral" in the function name.

This distinction is important for the thesis: CCR's review output is sensitive to local lexical context (names, comments, surrounding code) but entirely insensitive to the longitudinal feedback context accumulated across review sessions. These are two fundamentally different types of context, and only the latter supports genuine adaptation.

6. PR2 Results (Branch: test/ccr-feedback)

6.1 Setup

PR1 was merged into main along with the Copilot instruction files, leaving the code issues intentionally unresolved as research artifacts. PR2 was opened from a new branch (test/ccr-feedback) off main. The instruction files were therefore active in the repository when CCR reviewed PR2. The developer had dismissed the `Math.random()` comment in PR1 using the thumbs-down reaction prior to merging.'

6.2 Commits in PR2

#	Commit Message	Purpose	Pattern re-introduced
1	feat: add advanced product search with filters	Cross-PR adaptation test — same ReDoS pattern as PR1's <code>searchUsers</code> , in a new function	Unescaped <code>\$regex</code> on raw <code>q</code> input in <code>searchProducts</code>
2	feat: add product review invitation link generator	False positive dismissal test — same <code>Math.random()</code> pattern the developer thumbs-downed in PR1	<code>Math.random().toString(36)</code> for token generation in <code>generateReviewInvite</code>

6.3 PR2 CCR Comments

CCR reviewed 2 out of 2 changed files and generated 7 comments.

File	Issue Flagged	Classification	Notes
productController.js	ReDoS — <code>q</code> used directly as <code>\$regex</code> without escaping	Repeated from PR1	Flagged again with no reference to the identical PR1 comment on <code>searchUsers</code> . CCR cited the instruction file ("see <code>.github/copilot-instructions.md:32</code> ") rather than its own prior review — confirming it uses static context but has no cross-PR review memory.
productController.js	<code>Math.random()</code> not suitable for security tokens	Repeated from PR1 (dismissed)	Flagged again despite the developer's explicit thumbs-down dismissal in PR1. Near-identical wording. No acknowledgement of prior dismissal. Feedback signal completely ignored.
productController.js	<code>minPrice/maxPrice</code> coerced with <code>Number()</code> without NaN validation — malformed input silently produces unexpected query behaviour	New catch	Not planted intentionally. CCR identified an additional correctness issue in <code>searchProducts</code> .
productController.js	Unbounded <code>Product.find(filter)</code> — no pagination or result limit, inconsistent with <code>getProducts</code> which uses <code>PAGINATION_LIMIT</code>	New catch (context-aware)	Notably, CCR referenced <code>PAGINATION_LIMIT</code> from the instruction file to flag the inconsistency. This shows instruction-file context improving

			cross-function consistency awareness.
productController.js	Generated invite token never persisted or validated — the invite URL cannot actually authorise anything	New catch	Logical dead-feature issue detectable within the function.
productController.js	process.env.FRONTEND_URL used without checking if it is set — returns broken links if misconfigured	New catch	Defensive coding pattern, not project-specific.
productRoutes.js	PR scope comment — review-invite endpoint not reflected in PR title/description	Meta comment	CCR flagged a process issue (PR hygiene), not a code issue. Demonstrates awareness of PR description quality.

6.4 Adaptation Measurements — Final Results

Dimension	PR1 Initial	PR1 Re-review	PR2 (with instructions)
Hardcoded threshold caught	No	Yes (instruction)	N/A — not in PR2 diff
Password leak in analytics caught	No	Yes (instruction)	N/A — not in PR2 diff
N+1 identified as performance issue	No	Yes (instruction)	N/A — not in PR2 diff
Date timezone issue caught	No	Yes (instruction)	N/A — not in PR2 diff
ReDoS flagged (same pattern as PR1)	Yes	Yes (repeated)	Yes — repeated, no cross-PR reference
ReDoS comment references prior PR1 flag	N/A	N/A	No — cited instruction file only
Math.random() flagged despite PR1 thumbs-down	N/A	Yes (flagged, dismissed)	Yes — repeated, dismissal ignored
Any reference to PR1 feedback in PR2 comments	N/A	N/A	No — zero cross-PR awareness

6.5 Key Finding: Static vs Dynamic Context

PR2 establishes a definitive boundary in CCR's context model:

- Static context (instruction files, code comments): actively consulted — CCR cited `.github/copilot-instructions.md:32` by line number in its ReDoS comment, and referenced `PAGINATION_LIMIT` from the instructions when flagging the unbounded query.
- Dynamic context (review history, developer feedback, prior comments): not available — the thumbs-down dismissal, the unresolved PR1 comments, and the identical prior flag on `searchUsers` were all invisible to CCR.

This distinction is the empirical foundation of the thesis argument: an adaptive system must treat review history as a first-class context source alongside static documentation.

7. PR3 Results (Branch: test/ccr-feedback)

7.1 Setup

PR2 was merged into main. A new commit was added to the same branch (test/ccr-feedback) and PR3 was opened against main. At this point the developer had dismissed the `Math.random()` pattern twice — once via thumbs-down (PR1) and once via an explicit text comment explaining the false positive rationale (PR2). The instruction files were active in the repository.

7.2 Commit in PR3

Commit Message	Purpose	Pattern re-introduced
feat: add user referral code generation	Tests whether an explicit text dismissal in PR2 suppresses the <code>Math.random()</code> flag in PR3	<code>Math.random().toString(36)</code> in <code>generateReferralCode</code> — clearly non-security context (marketing referral code)

7.3 PR3 CCR Comments

CCR reviewed 2 out of 2 changed files and generated 3 comments.

File	Issue Flagged	Classification	Notes
userController.js	<code>Math.random()</code> used for referral code — guessable and collision-prone for referral systems; no uniqueness check or persistence	Repeated (third time) — both prior dismissals ignored	No reference to the PR1 thumbs-down or the PR2 explicit text dismissal. Wording evolved to "referral systems... guessable and collision-prone" — driven by the function name <code>generateReferralCode</code> , not by feedback learning. See Section 5.9.
userController.js	Unnecessary <code>User.findById(req.user._id)</code> — <code>protect</code> middleware already loads the user into <code>req.user</code> ; redundant DB round-trip on every call	New catch	Not planted. CCR identified a genuine performance issue by recognising the project's <code>protect</code> middleware pattern — a cross-file awareness enabled by the instruction files and codebase context.
userRoutes.js	<code>POST /api/users/referral</code> has no side effects — controller only returns a generated code without persisting it; semantically should be GET	New catch	REST semantics correctness issue. Not project-specific — detectable from HTTP convention knowledge alone.

7.4 Key Finding: Explicit Text Dismissal Also Ignored

The PR3 result confirms that the explicit text dismissal in PR2 — which named the false positive rationale, the specific use case, and the reason `crypto.randomBytes()` was unnecessary — had precisely the same effect as the PR1 thumbs-down: none. CCR flagged the pattern a third time, with no reference to either prior dismissal.

This rules out the possibility that the failure in PR1 was simply due to the ambiguity of a thumbs-down signal. A clear, developer-authored natural language explanation of why the flag is wrong in this project's context is equally invisible to CCR. The limitation is architectural: CCR's review model has no access to any prior review session data, regardless of how that feedback was expressed.

8. Summary of Findings

Metric	PR1 Initial	PR1 Re-review	PR2	PR3
Files reviewed	8 of 9	10 of 11	2 of 2	2 of 2
Total inline comments	13	10	7	3
Category A issues caught	5/5 (100%)	5/5 (100%)	2/2 (100%)	1/1 (100%)
Context-dependent issues caught (no instructions)	0/4 (0%)	4/4 (100%)	1/1 via instruction	1/1 via middleware context
N+1 identified as performance anti-pattern	No	Yes (instruction)	N/A	N/A
Math.random() flagged (cumulative)	Yes (1st)	N/A	Yes (2nd — thumbs-down ignored)	Yes (3rd — text dismissal ignored)
Dismissal method applied	Thumbs-down	N/A	Explicit text comment	—
Prior dismissal acknowledged by CCR	N/A	N/A	No	No
Wording changed due to feedback	N/A	N/A	No	No (changed due to function name only)
Cross-PR review history referenced	N/A	N/A	No	No
Auto-triggered on new commit	Yes (PR open)	No (manual)	Yes (PR open)	Yes (PR open)

Overall Conclusion

Across four review rounds and three pull requests, the following findings are established:

- **Generic issues (Category A):** CCR achieves 100% recall consistently, with accurate diagnosis and actionable, ready-to-apply code suggestions. This is its strongest capability.
- **Context-dependent issues (Category B):** CCR misses these entirely without explicit instruction files. With instructions, recall improves to 100% on targeted issues — but only for conventions manually pre-documented by the developer.
- **Dynamic feedback adaptation (RQ2):** CCR has no capacity for cross-PR learning. Neither implicit signals (thumbs-down) nor explicit text dismissals have any effect on future review behaviour. The

Math.random() pattern was flagged identically three times despite two distinct dismissal methods. Each PR is reviewed in complete isolation from all prior review history.

- **Lexical vs longitudinal context:** CCR adapts its wording to local code context (function names, surrounding patterns) but not to longitudinal feedback history. These are fundamentally different types of context, and only the latter supports genuine adaptation.
- **Static vs dynamic context boundary:** CCR actively consults static documents (instruction files, code comments, current diff) but cannot consume dynamic context (review history, prior comments, developer feedback signals). This boundary defines the gap that the proposed adaptive system addresses.

Experiment complete across PR1 (initial + re-review with instructions), PR2 (cross-PR adaptation test), and PR3 (explicit text dismissal test). All three research questions have been addressed with empirical evidence.